

第 6 章

阈值处理

阈值处理是指剔除图像内像素值高于一定值或者低于一定值的像素点。例如，设定阈值为 127，然后：

- 将图像内所有像素值大于 127 的像素点的值设为 255。
- 将图像内所有像素值小于或等于 127 的像素点的值设为 0。

通过上述方式能够得到一幅二值图像，如图 6-1 所示，按照上述阈值处理方式将一幅灰度图像处理为一幅二值图像，有效地实现了前景和背景的分离。

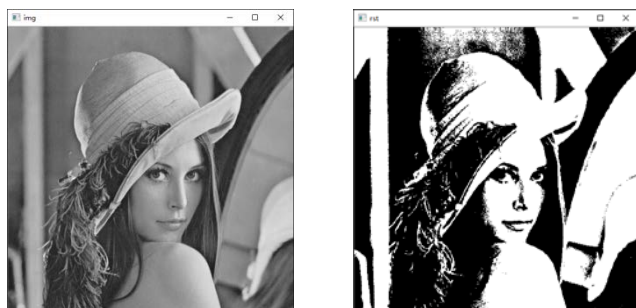


图 6-1 阈值处理示例

OpenCV 提供了函数 `cv2.threshold()` 和函数 `cv2.adaptiveThreshold()`，用于实现阈值处理。

6.1 threshold 函数

OpenCV 3.0 使用 `cv2.threshold()` 函数进行阈值化处理，该函数的语法格式为：

```
retval, dst = cv2.threshold( src, thresh, maxval, type )
```

式中：

- `retval` 代表返回的阈值。
- `dst` 代表阈值分割结果图像，与原始图像具有相同的大小和类型。
- `src` 代表要进行阈值分割的图像，可以是多通道的，8 位或 32 位浮点型数值。
- `thresh` 代表要设定的阈值。
- `maxval` 代表当 `type` 参数为 `THRESH_BINARY` 或者 `THRESH_BINARY_INV` 类型时，需要设定的最大值。
- `type` 代表阈值分割的类型，具体类型值如表 6-1 所示。



表 6-1 阈值分割类型

类型	定义
cv2.THRESH_BINARY	$\text{dst}(x,y) = \begin{cases} \text{maxval}, & \text{src}(x,y) > \text{thresh} \\ 0, & \text{其他情况} \end{cases}$
cv2.THRESH_BINARY_INV	$\text{dst}(x,y) = \begin{cases} 0, & \text{src}(x,y) > \text{thresh} \\ \text{maxval}, & \text{其他情况} \end{cases}$
cv2.THRESH_TRUNC	$\text{dst}(x,y) = \begin{cases} \text{thresh}, & \text{src}(x,y) > \text{thresh} \\ 0, & \text{其他情况} \end{cases}$
cv2.THRESH_TOZERO_INV	$\text{dst}(x,y) = \begin{cases} 0, & \text{src}(x,y) > \text{thresh} \\ \text{src}(x,y), & \text{其他情况} \end{cases}$
cv2.THRESH_TOZERO	$\text{dst}(x,y) = \begin{cases} \text{src}(x,y), & \text{src}(x,y) > \text{thresh} \\ 0, & \text{其他情况} \end{cases}$
cv2.THRESH_MASK	掩码
cv2.THRESH_OTSU	标记, 使用 Otsu 算法时的可选阈值参数
cv2.THRESH_TRIANGLE	标记, 使用 Triangle 算法时的可选阈值参数

上述公式相对抽象，可以将其可视化，具体如图 6-2 所示。

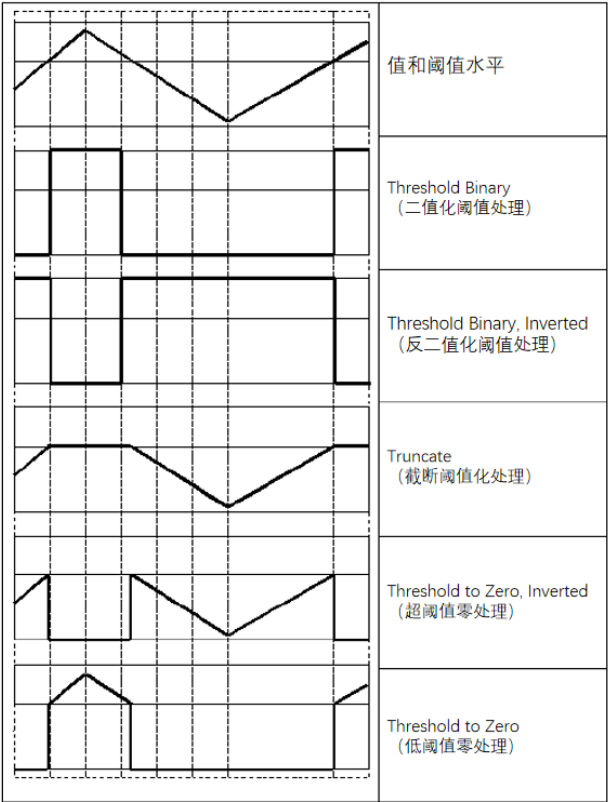


图 6-2 阈值分割类型可视化表示

6.1.1 二值化阈值处理（cv2.THRESH_BINARY）

二值化阈值处理会将原始图像处理为仅有两个值的二值图像，其示意图如图 6-3 所示。其针对像素点的处理方式为：

- 对于灰度值大于阈值`thresh`的像素点，将其灰度值设定为最大值。
- 对于灰度值小于或等于阈值`thresh`的像素点，将其灰度值设定为 0。

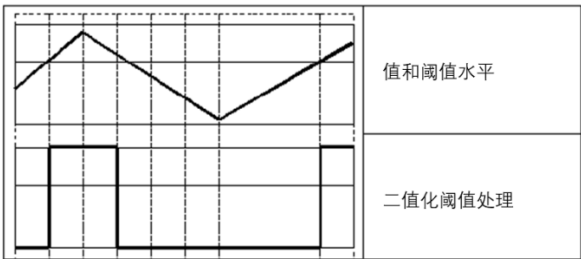


图 6-3 二值化阈值处理

如果使用表达式表示，其目标值的产生规则为：

$$\text{dst}(x,y) = \begin{cases} \text{maxval}, & \text{src}(x,y) > \text{thresh} \\ 0, & \text{其他情况} \end{cases}$$

式中，`thresh`是选定的特定阈值。

在 8 位图像中，最大值是 255。因此，在对 8 位灰度图像进行二值化时，如果将阈值设定为 127，那么：

- 所有大于 127 的像素点会被处理为 255。
- 其余值会被处理为 0。

为了方便，在后续说明中，我们都以 8 位图像为例，即像素值最大值为 255。

【例 6.1】使用函数 `cv2.threshold()` 对数组进行二值化阈值处理，观察处理结果。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
t,rst=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
print("img=\n",img)
print("t=",t)
print("rst=\n",rst)
```

运行程序，结果如下所示：

```
img=
[[184 204 23 247 118]
 [173 107 120 69 209]
 [231 218 42 211 108]
 [133 125 29 191 198]]
t= 127.0
rst=
[[255 255 0 255 0]
 [255 0 0 0 255]
 [255 255 0 255 0]
 [255 0 0 255 255]]
```

【例 6.2】 使用函数 `cv2.threshold()` 对图像进行二值化阈值处理。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp")
t,rst=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
cv2.imshow("img",img)
cv2.imshow("rst",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-4 所示，左图是原始图像，右图是二值化阈值处理结果。

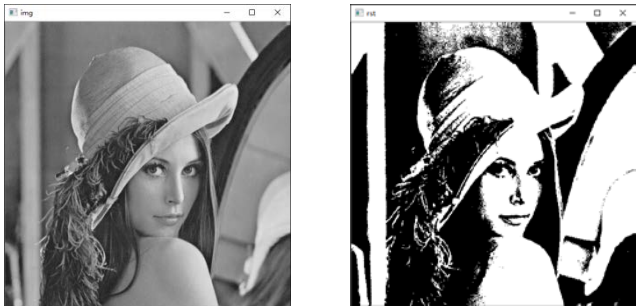


图 6-4 二值化阈值处理结果

6.1.2 反二值化阈值处理（`cv2.THRESH_BINARY_INV`）

反二值化阈值处理的结果也是仅有两个值的二值图像，与二值化阈值处理的区别在于，二者对像素值的处理方式不同。反二值化阈值处理针对像素点的处理方式为：

- 对于灰度值大于阈值的像素点，将其值设定为 0。
- 对于灰度值小于或等于阈值的像素点，将其值设定为 255。

反二值化阈值处理方式的示意图如图 6-5 所示。

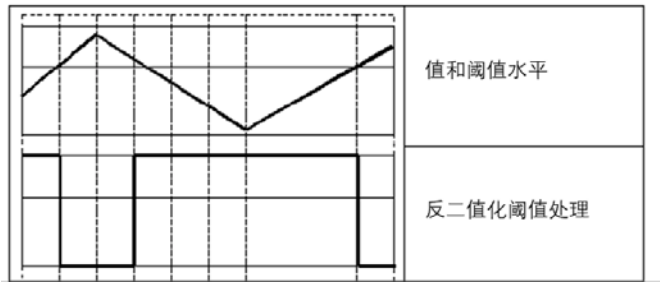


图 6-5 反二值化阈值处理

如果使用表达式来表示，其目标值的产生规则为：

$$dst(x,y) = \begin{cases} 0, & src(x,y) > thresh \\ maxval, & \text{其他情况} \end{cases}$$

式中，`thresh` 是选定的阈值。

【例 6.3】 使用函数 `cv2.threshold()` 对数组进行反二值化阈值处理，观察处理结果。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
t,rst=cv2.threshold(img,127,255,cv2.THRESH_BINARY_INV)
print("img=\n",img)
print("t=",t)
print("rst=\n",rst)
```

运行程序，结果如下所示：

```
img=
[[ 56  64 150  48  41]
 [108 165 112 213 110]
 [122 244  10 213  46]
 [247  30  90   0 26]]
t= 127.0
rst=
[[255 255   0 255 255]
 [255   0 255   0 255]
 [255   0 255   0 255]
 [  0 255 255 255 255]]
```

【例 6.4】 使用函数 `cv2.threshold()` 对图像进行反二值化阈值处理。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp")
t,rst=cv2.threshold(img,127,255,cv2.THRESH_BINARY_INV)
cv2.imshow("img",img)
cv2.imshow("rst",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-6 所示，其中左图是原始图像，右图是反二值化阈值处理结果。

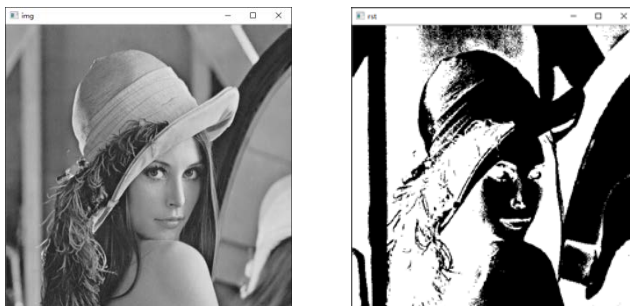


图 6-6 反二值化阈值处理结果

6.1.3 截断阈值化处理 (cv2.THRESH_TRUNC)

截断阈值化处理会将图像中大于阈值的像素点的值设定为阈值，小于或等于该阈值的像素点的值保持不变。这种处理方式的示意图如图 6-7 所示。

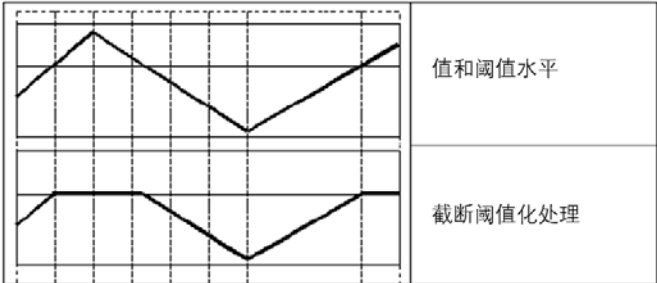


图 6-7 截断阈值化处理

例如，阈值选取为 127，则截断阈值化处理时：

- 对于像素值大于 127 的像素点，其像素值将被设定为 127。
- 对于像素值小于或等于 127 的像素点，其像素值将保持改变。

如果使用表达式表示，那么其目标值的产生规则为：

$$dst(x,y) = \begin{cases} thresh, & src(x,y) > thresh \\ 0, & \text{其他情况} \end{cases}$$

【例 6.5】 使用函数 cv2.threshold()对数组进行截断阈值化处理，观察处理结果。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
t,rst=cv2.threshold(img,127,255,cv2.THRESH_TRUNC)
print("img=\n",img)
print("t=",t)
print("rst=\n",rst)
```

运行程序，结果如下所示：

```
img=
[[190  60 146  22  22]
 [253  49  63 180 113]
 [ 27  64 148  44   7]
 [ 11 100 249  47 239]]
t= 127.0
rst=
[[127  60 127  22  22]
 [127  49  63 127 113]
 [ 27  64 127  44   7]
 [ 11 100 127  47 127]]
```

【例 6.6】 使用函数 cv2.threshold()对图像进行截断阈值化处理。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp")
t,rst=cv2.threshold(img,127,255,cv2.THRESH_TRUNC)
cv2.imshow("img",img)
cv2.imshow("rst",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-8 所示，其中左图是原始图像，右图是截断阈值化处理结果。

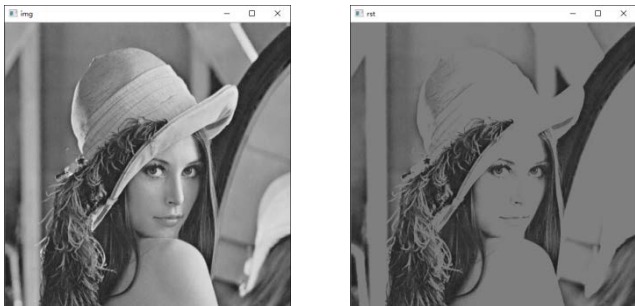


图 6-8 截断阈值化处理结果

6.1.4 超阈值零处理 (cv2.THRESH_TOZERO_INV)

超阈值零处理会将图像中大于阈值的像素点的值处理为 0，小于或等于该阈值的像素点的值保持不变。即先选定一个阈值，然后对图像做如下处理：

- 对于像素值大于阈值的像素点，其像素值将被处理为 0。
- 对于像素值小于或等于阈值的像素点，其像素值将保持不变。

超阈值零处理的工作原理如图 6-9 所示。

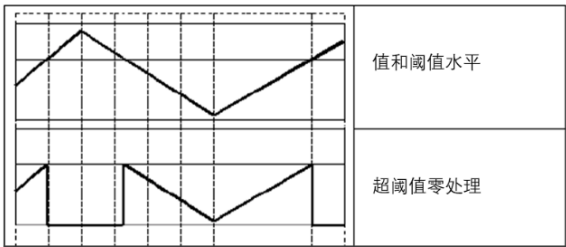


图 6-9 超阈值零处理

例如，阈值选取为 127，则：

- 对于像素值大于 127 的像素点，其值将被设定为 0。
- 对于像素值小于或等于 127 的像素点，其值将保持改变。

如果使用表达式表示，其目标值的产生规则为：

$$dst(x,y) = \begin{cases} 0, & src(x,y) > thresh \\ src(x,y), & \text{其他情况} \end{cases}$$

【例 6.7】使用函数 `cv2.threshold()` 对数组进行超阈值零处理，观察处理结果。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
t,rst=cv2.threshold(img,127,255,cv2.THRESH_TOZERO_INV)
print("img=\n",img)
print("t=",t)
print("rst=\n",rst)
```

运行程序，结果如下所示：

```
img=
[[ 16  68 231  94  82]
 [112 195 239 127  71]
 [ 67  47 240 107 227]
 [144  51 130 207 164]]
t= 127.0
rst=
[[ 16  68   0  94  82]
 [112   0   0 127  71]
 [ 67  47   0 107   0]
 [  0  51   0   0   0]]
```

【例 6.8】使用函数 `cv2.threshold()` 对图像进行超阈值零处理。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp")
t,rst=cv2.threshold(img,127,255,cv2.THRESH_TOZERO_INV)
cv2.imshow("img",img)
cv2.imshow("rst",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-10 所示，其中左图是原始图像，右图是超阈值零处理结果。

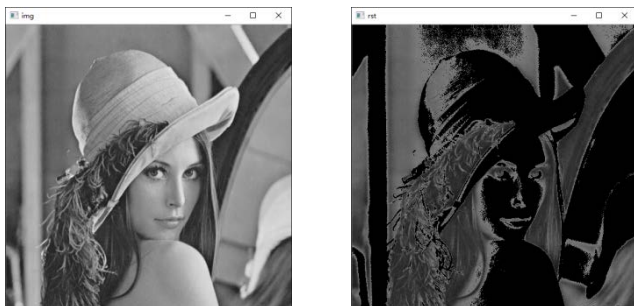


图 6-10 超阈值零处理结果

6.1.5 低阈值零处理 (cv2.THRESH_TOZERO)

低阈值零处理会将图像中小于或等于阈值的像素点的值处理为 0，大于阈值的像素点的值保持不变。即先选定一个阈值，然后对图像做如下处理：

- 对于像素值大于阈值的像素点，其值将保持不变。
- 对于像素值小于或等于阈值的像素点，其值将被处理为 0。

其示意图如图 6-11 所示。

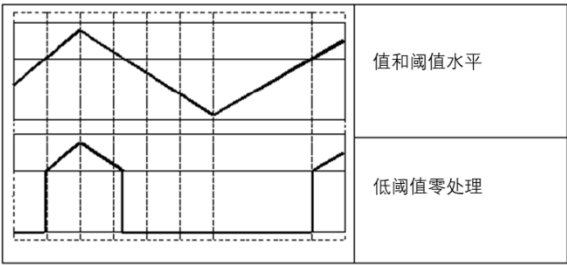


图 6-11 低阈值零处理

例如，阈值选取为 127，则：

- 对于像素值大于 127 的像素点，其像素值将保持改变。
- 对于像素值小于或等于 127 的像素点，其像素值将被设定为 0。

如果使用表达式表示，其目标值的产生规则为：

$$\text{dst}(x,y) = \begin{cases} \text{src}(x,y), & \text{src}(x,y) > \text{thresh} \\ 0, & \text{其他情况} \end{cases}$$

【例 6.9】使用函数 cv2.threshold()对数组进行低阈值零处理，观察处理结果。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
t,rst=cv2.threshold(img,127,255,cv2.THRESH_TOZERO)
print("img=\n",img)
print("t=",t)
print("rst=\n",rst)
```

运行程序，结果如下所示：

```
img=
[[ 73  57 135  10   8]
 [127 162 110 217  54]
 [203  99 205  97 127]
 [ 14 100  83 226  71]]
t= 127.0
rst=
[[ 0  0 135  0  0]
 [ 0 162  0 217  0]
```

```
[203  0 205  0  0]
[  0  0  0 226  0]]
```

【例 6.10】 使用函数 `cv2.threshold()` 对图像进行低阈值零处理。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp")
t,rst=cv2.threshold(img,127,255,cv2.THRESH_TOZERO)
cv2.imshow("img",img)
cv2.imshow("rst",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-12 所示，其中左图是原始图像，右图是低阈值零处理结果。

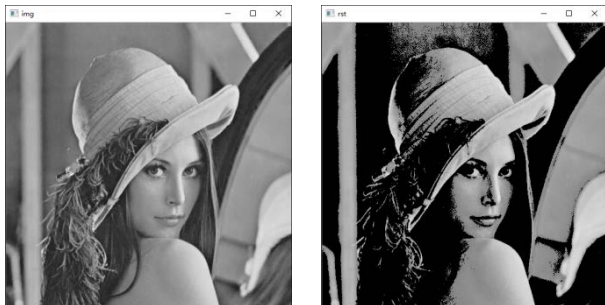


图 6-12 低阈值零处理结果

6.2 自适应阈值处理

对于色彩均衡的图像，直接使用一个阈值就能完成对图像的阈值化处理。但是，有时图像的色彩是不均衡的，此时如果只使用一个阈值，就无法得到清晰有效的阈值分割结果图像。

有一种改进的阈值处理技术，其使用变化的阈值完成对图像的阈值处理，这种技术被称为自适应阈值处理。在进行阈值处理时，自适应阈值处理的方式通过计算每个像素点周围临近区域的加权平均值获得阈值，并使用该阈值对当前像素点进行处理。与普通的阈值处理方法相比，自适应阈值处理能够更好地处理明暗差异较大的图像。

OpenCV 提供了函数 `cv2.adaptiveThreshold()` 来实现自适应阈值处理，该函数的语法格式为：

```
dst = cv.adaptiveThreshold( src, maxValue, adaptiveMethod, thresholdType,
blockSize, C )
```

式中：

- `dst` 代表自适应阈值处理结果。
- `src` 代表要进行处理的原始图像。需要注意的是，该图像必须是 8 位单通道的图像。
- `maxValue` 代表最大值。
- `adaptiveMethod` 代表自适应方法。
- `thresholdType` 代表阈值处理方式，该值必须是 `cv2.THRESH_BINARY` 或者



`cv2.THRESH_BINARY_INV` 中的一个。

- `blockSize` 代表块大小。表示一个像素在计算其阈值时所使用的邻域尺寸，通常为 3、5、7 等。
- `C` 是常量。

函数 `cv2.adaptiveThreshold()` 根据参数 `adaptiveMethod` 来确定自适应阈值的计算方法，函数包含 `cv2.ADAPTIVE_THRESH_MEAN_C` 和 `cv2.ADAPTIVE_THRESH_GAUSSIAN_C` 两种不同的方法。这两种方法都是逐个像素地计算自适应阈值，自适应阈值等于每个像素由参数 `blockSize` 所指定邻域的加权平均值减去常量 `C`。两种不同的方法在计算邻域的加权平均值时所采用的方式不同：

- `cv2.ADAPTIVE_THRESH_MEAN_C`：邻域所有像素点的权重值是一致的。
- `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`：与邻域各个像素点到中心点的距离有关，通过高斯方程得到各个点的权重值。

【例 6.11】 对一幅图像分别使用二值化阈值函数 `cv2.threshold()` 和自适应阈值函数 `cv2.adaptiveThreshold()` 进行处理，观察处理结果的差异。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("computer.jpg",0)
t1,thd=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
athdMEAN=cv2.adaptiveThreshold(img,255,cv2.ADAPTIVE_THRESH_MEAN_C,cv2.THRESH_BINARY,5,3)
athdGAUS=cv2.adaptiveThreshold(img,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,5,3)
cv2.imshow("img",img)
cv2.imshow("thd",thd)
cv2.imshow("athdMEAN",athdMEAN)
cv2.imshow("athdGAUS",athdGAUS)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-13 所示，其中：

- 图(a)是原始图像。
- 图(b)是二值化阈值处理结果。
- 图(c)是自适应阈值采用方法 `cv2.ADAPTIVE_THRESH_MEAN_C` 的处理结果。
- 图(d)是自适应阈值采用方法 `cv2.ADAPTIVE_THRESH_GAUSSIAN_C` 的处理结果。

通过对比普通的阈值处理与自适应阈值处理可以发现，自适应阈值处理保留了更多的细节信息。在一些极端情况下，普通的阈值处理会丢失大量的信息，而自适应阈值处理可以得到效果更好的二值图像。

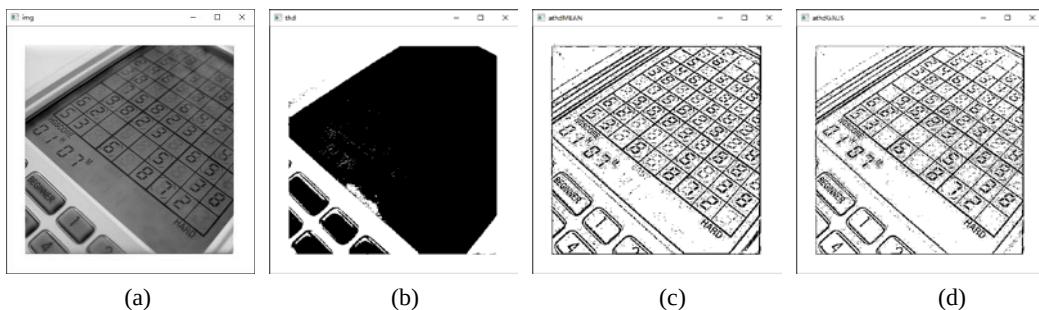


图 6-13 【例 6.11】程序的运行结果

6.3 Otsu 处理

在使用函数 `cv2.threshold()` 进行阈值处理时，需要自定义一个阈值，并以此阈值作为图像阈值处理的依据。通常情况下处理的图像都是色彩均衡的，这时直接将阈值设为 127 是比较合适的。

但是，有时图像灰度级的分布是不均衡的，如果此时还将阈值设置为 127，那么阈值处理的结果就是失败的。例如，有一个图像 `img`，里面的像素值为：

```
[[123 123 123 123 123]
 [123 123 123 123 123]
 [123 123 126 126 126]
 [123 123 126 126 126]
 [123 123 126 126 126]]
```

此时，如果仍然以 127 作为阈值，那么阈值处理结果是：

```
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
```

很显然，这不是我们想要的结果。我们可以观察到，对于 `img`，如果以阈值 125 进行分割，可以得到较好的结果：

```
[[ 0  0  0  0  0]
 [ 0  0  0  0  0]
 [ 0  0 255 255 255]
 [ 0  0 255 255 255]
 [ 0  0 255 255 255]]
```

但是，实际处理的图像往往是很复杂的，不太可能像上述 `img` 那样，一眼就观察出最合适的阈值。如果一个个去尝试，工作量无疑是巨大的。

Otsu 方法能够根据当前图像给出最佳的类间分割阈值。简而言之，Otsu 方法会遍历所有可能阈值，从而找到最佳的阈值。

在 OpenCV 中，通过在函数 `cv2.threshold()` 中对参数 `type` 的类型多传递一个参数

“cv2.THRESH_OTSU”，即可实现 Otsu 方式的阈值分割。

需要说明的是，在使用 Otsu 方法时，要把阈值设为 0。此时的函数 cv2.threshold() 会自动寻找最优阈值，并将该阈值返回。例如，下面的语句让函数 cv2.threshold() 采用 Otsu 方法进行阈值分割：

```
t,otsu=cv2.threshold(img,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
```

与普通阈值分割的不同之处在于：

- 参数 type 增加了一个参数值 “cv2.THRESH_OTSU”。
- 设定的阈值为 0。
- 返回值 t 是 Otsu 方法计算得到并使用的最优阈值。

需要注意，如果采用普通的阈值分割，返回的阈值就是设定的阈值。例如下面的语句设定阈值为 127，所以最终返回的就是 t=127。

```
t,thd=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
```

【例 6.12】 测试 Otsu 阈值处理的实现。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img = np.zeros((5,5),dtype=np.uint8)
img[0:6,0:6]=123
img[2:6,2:6]=126
print("img=\n",img)
t1,thd=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
print("thd=\n",thd)
t2,otsu=cv2.threshold(img,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
print("otsu=\n",otsu)
```

运行程序，结果如下所示。

```
img=
[[123 123 123 123 123]
 [123 123 123 123 123]
 [123 123 126 126 126]
 [123 123 126 126 126]
 [123 123 126 126 126]]
thd=
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
otsu=
[[ 0  0  0  0  0]
 [ 0  0  0  0  0]
 [ 0  0 255 255 255]]
```

```
[ 0  0 255 255 255]
[ 0  0 255 255 255]]
```

【例 6.13】 分别对一幅图像进行普通的二值化阈值处理和 Otsu 阈值处理，观察处理结果的差异。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("tiffany.bmp",0)
t1,thd=cv2.threshold(img,127,255,cv2.THRESH_BINARY)
t2,otsu=cv2.threshold(img,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
cv2.imshow("img",img)
cv2.imshow("thd",thd)
cv2.imshow("otus",otsu)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 6-14 所示，其中：

- 图(a)是原始图像。
- 图(b)是进行普通二值化阈值处理，以 127 为阈值的处理结果。
- 图(c)是二值化阈值处理采用参数 `cv2.THRESH_OTSU` 后的处理结果。

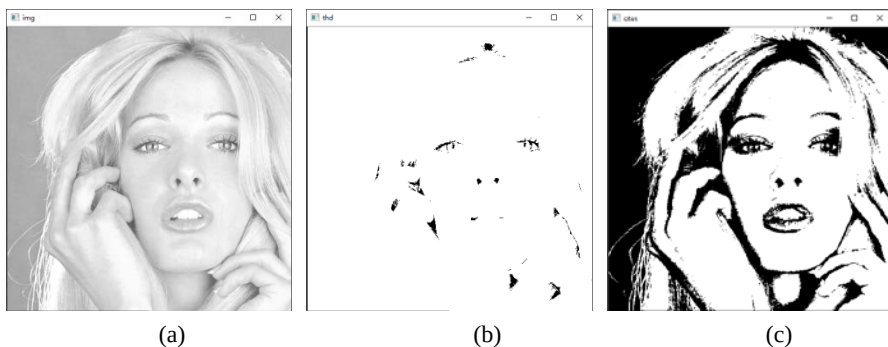


图 6-14 【例 6.13】程序的运行结果

在本例中，原始图像整体的亮度较高，即有较多的像素值都是大于 127 的，所以：

- 在使用 127 作为阈值进行普通的二值阈值化处理时，得到了大量的白色区域。
- 在使用 Otsu 方法进行处理时，因为通过计算采用了最优阈值，所以得到了较好的处理结果。